XBRAIN CAPSTONE · TASK FORCE 4 · AIO-03

Foresight Lens

Resource exhaustion prediction system for Tier 1 cloud services.

Lead time ≥ 15 min | Inference ~$0 cost | Model ewma-stl-v1

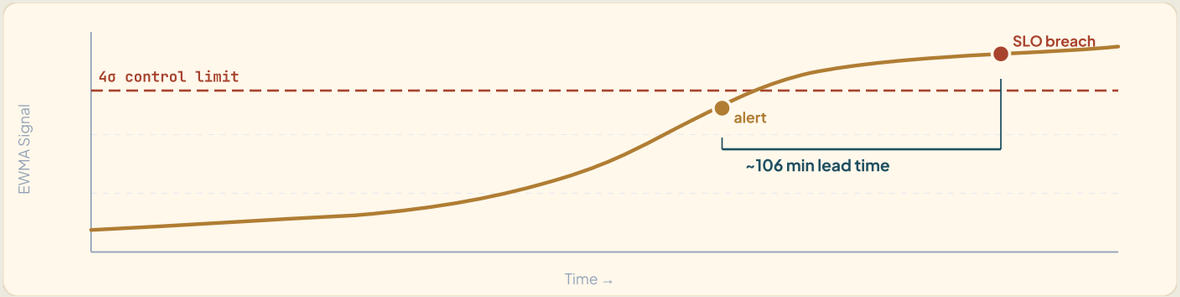


Fig. 1 — EWMA control chart issuing early alerts ~106 minutes before SLO breach.

02

CLIENT CONTEXT

When the monitor turns red, it's already too late

Traditional monitoring only triggers at 99% — leaving SREs with zero response time.

CLIENT CONTEXT

8 Core Requirements

- **Proactive prediction:** Must alert before actual system crash (Lead time ≥ 15 mins).
- **High Precision:** FP rate ≤ 12%, detecting ≥ 80% of drifts.
- **Cost Control:** Total monthly AWS bill strictly under $200.
- **Multi-tenant Security:** Applied to 3 tier-1 services with isolated baselines and zero cross-tenant bleed.
- **No Auto-remediation:** AI strictly acts as an advisor (PURELY Predictive) without system intervention.
- **Per-service Baseline:** Train baselines per service (1 manual run) + ADRs defining retrain triggers.
- **Actionable Recommendation:** Must include 5 components (action verb, target, from → to, confidence, evidence).
- **Audit Log:** Log every prediction request, encrypted at rest.

≤ $200	Advise	3 svc
AWS BUDGET/MONTH	NO AUTO-REMEDIATION	PAYMENT · FRAUD · LEDGER

PROJECT BOUNDARY

Out of Scope (Client Confirmed)

Strict boundaries established from W11 to ensure focus and operational safety.

Functional Exclusions

- **Auto-remediation:** Predict & recommend only.
- **Cross-service RCA:** Per-service drift only.
- **Cost forecasting:** Transferred to Task Force 2.
- **Business Metrics:** Infra metrics only (no fraud rate, tx count).

Operational Exclusions

- **Auto-retrain pipeline:** Design ADR is sufficient.
- **Multi-region deployment:** Single region, DR design-only.
- **Production traffic mirror:** Local load test with k6/Locust.
- **Scale targets:** 99.5% SLO (not 99.99%) & exactly 3 Tier-1 services.

03

THE PROBLEM

Traditional monitoring alerts too late

Failure symptoms persist for over 90 minutes, yet systems only alert at the absolute brink.

THE PROBLEM

4 Factors Driving the Shift to Predictive Alerts

1. Silent Exhaustion

Incidents don't explode instantly; they silently decay (Memory leak, queue backlog) over 90+ minutes.

2. Alert Fatigue

A low Static threshold causes endless False Positives; a high one only alerts post-crash.

3. 24/7 Blindspot

Systems generate too many metrics. Nobody has the time to stare at 12 Grafana dashboards 24/7.

4. Dynamic Seasonality

Fintech traffic has stark day/night cycles. Relying on a fixed baseline is entirely meaningless.

HARSH REALITY

Illustration: Silent Exhaustion Incident

The system struggles for 3 continuous hours, yet the Static threshold remains completely blind.

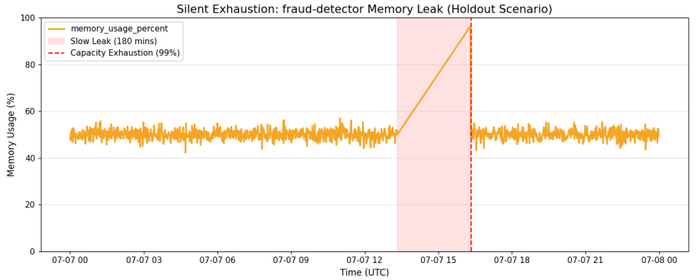


Fig. 2 — A memory leak creeping up silently for 180 minutes. The 99% mark (Static threshold) flashes red only upon crashing. Lead time = 0.

04

RESULTS

Early predictions, clear advice — no unauthorized actions

Strictly acting as an Advisor. The CDO platform retains ultimate decision-making authority.

MEASURABLE OUTCOMES (1/4)

106 Minutes Lead Time

Memory leak anomaly alerted 106 minutes before capacity exhaustion crash.
Exceeds the ≥ 15m requirement.

```
// evidence_lead_time.json

{
  "method": "Simulate gradual CPU drift → detect via EWMA...",
  ...
  "detected_at_minute": 734,
  "breach_at_minute": 840,
  "lead_time_minutes": 106,
  "requirement": "≥ 15 minutes",
  "pass": true
}
```

MEASURABLE OUTCOMES (2/4)

3 Isolated Tenants

Per-service baselines strictly maintain tenant isolation with statistical proof of pattern diversity.

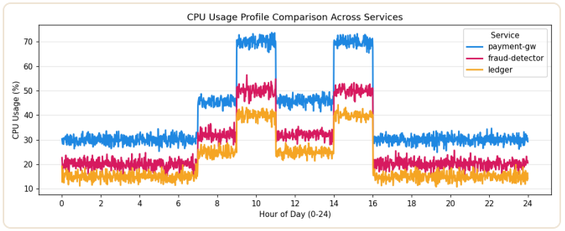


Fig — 3 services exhibiting entirely different daily traffic behaviors. A 1-size-fits-all static threshold is impossible.

```
// evidence_pattern_diversity.json

{
  "claim": "3 services exhibit distinctly different CPU patterns",
  "tests": {
    "anova": { "f_statistic": 3234.51, "p_value": 0.0, "significant": true },
    "peak_hour_analysis": {
      "payment-gw": 10,
      "ledger": 1,
      "fraud-detector": 13
    }
  },
  "conclusion": "Static thresholds would trigger massive False Positives."
}
```

EVIDENCE REFERENCE

evidence_pattern_diversity.json

MEASURABLE OUTCOMES (3/4)

$0 Token Cost

In-house statistical model costs zero LLM tokens. Complete AWS footprint runs at ~$53/month.

```
// evidence_cost.json

{
  "services": {
    "ECS Fargate": { "cost": "$36.00/month" },
    "Application Load Balancer": { "cost": "$16.00/month" },
    "API_Gateway_private": { "cost": "$0" },
    "S3_audit_and_baselines": { "cost": "$1.00/month" }
  },
  "total_estimated_monthly": "$53.00",
  "budget_cap": "$200/month",
  "conclusion": "Using ECS Fargate is cost-effective (~$53/mo)... Inference token cost is $0."
}
```

MEASURABLE OUTCOMES (4/4)

5 Action Components

Alerts are bundled with specific action verbs, targets, transitions, confidence scores, and evidence.

```
// evidence_recommendation.json

{
  "requirement_met": "action verb + target + from→to + confidence + evidence link",
  "test_cases": [
    {
      "metric": "cpu_usage_percent",
      "recommendation": "SCALE_UP (Action) RDS Database (Target) from db.r5.large to db.r5.xlarge (From→To). Confidence: 0.85 (runtime-computed). Evidence: https://obs.internal/tnt-1?metric=cpu_usage_percent"
    },
    ...
  ]
}
```

05

METHODOLOGY & RATIONALE

Why we built it this way

Behind every design decision is a rigorous defense backed by hard evidence.

ALGORITHMIC RATIONALE · PRE-FLIGHT

Synthetic Baseline Generation

How we generated statistically rigorous training data and holdout sets for the STL model.

7-Day Telemetry Simulation

To train our models without access to production data, we simulated 7 days of 1-minute granularity telemetry per service.

- **Base Seasonal Trend:** `daily_shape()` applies a "business-hours double-hump load curve" (peaking at 9–11 AM and 2–4 PM).
- **Noise Injection:** Gaussian white noise `RNG.normal(0, sigma)` added to simulate real-world traffic fluctuations.
- **STL Extraction:** We then ran STL (period=1440) on the clean data (Days 0–5) to extract the seasonal profile and residual standard deviation.

Holdout Set & Honest Evaluation

We isolated **Day 6** as our Holdout evaluation set and injected 4 specific anomalies to measure detection speed:

- **Gradual Drift:** CPU ramps from 40% to 94% over 90 mins.
- **Sudden Spike:** Latency jumps +600ms for 20 mins.
- **Slow Leak:** Memory creeps from 50% to 96% over 180 mins.
- **Noisy FP Trap:** Massive queue depth variance but *no real mean shift* (Detector must stay silent).

Evidence: scripts/train_baseline.py

ALGORITHMIC RATIONALE · STEP 0A

Grounding tests in Historical Outages

We didn't invent random test cases. We mapped our Multi-Root-Cause Injection framework directly to the Client's exact pain points from the last 3 months.

CLIENT BRIEF (W11)

"RDS CPU bò lên 100% trong 90 phút..."

Test Vector 1

CPU Exhaustion

CLIENT BRIEF (W11)

"...trước khi connection pool exhaust."

Test Vector 2

Connection Pool Leak

CLIENT BRIEF (W11)

"Queue worker backlog âm thầm 6×..."

Test Vector 3

Queue Backlog

CLIENT BRIEF (W11)

"Memory leak dẫn đến OOM..."

Test Vector 4

Memory Leak

EVIDENCE REFERENCE

`TF4_FORESIGHT_LEARNER.md (Client Brief W11)`

ALGORITHMIC RATIONALE · STEP 0B

Visualizing the Lagging Trap

Across all simulated historical outages, **API Latency** and **Queue Depth** consistently break bounds long before CPU or Memory.

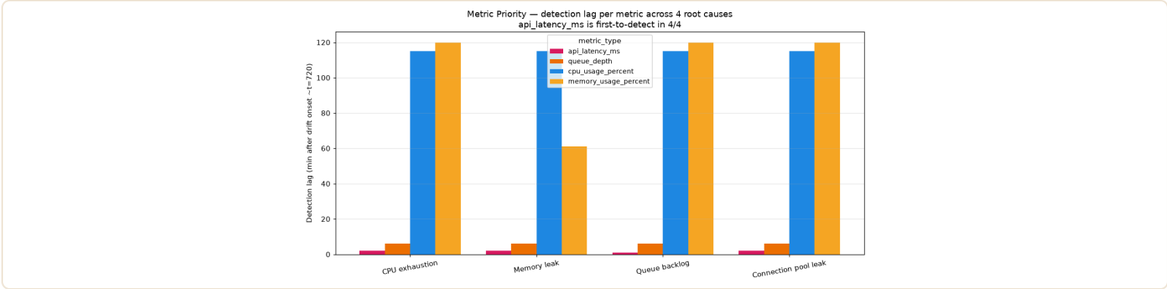


Fig 1 — Detection timeline. Latency detects at minute 722, Memory only flags at 781.

Metric Priority Assessment

Ranking the metrics based on detection speed across all fault vectors.

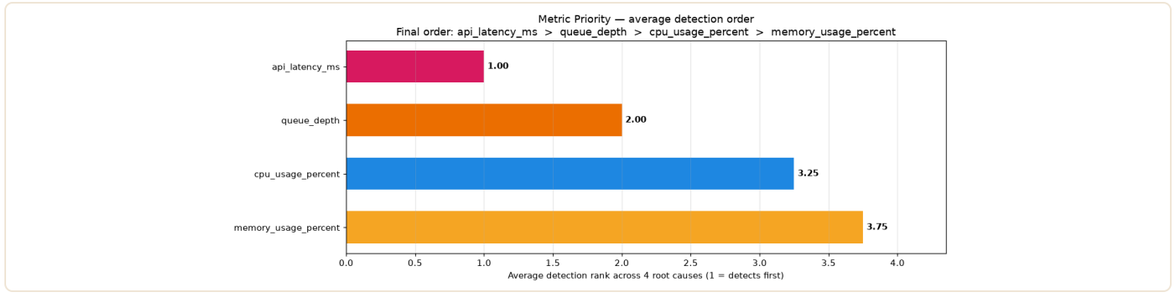


Fig 2 — Overall Metric Priority Score.

Mathematical Proof of Priority

We aggregated detection ranks across all 4 root causes. The result completely ruled out CPU and Memory as viable early-warning signals.

```
// evidence_metric_priority.json

{
  "method": "Multi-root-cause injection: 4 different failure modes",
  "circular_reasoning_fix": "Previous method injected only CPU drift. Now inject 4 independent root
causes.",
  "root_causes": [
    { "root_cause": "CPU exhaustion" },
    { "root_cause": "Memory leak" },
    { "root_cause": "Queue backlog" },
    { "root_cause": "Connection pool leak" }
  ],
  "average_rank": {
    "api_latency_ms": 1.0,
    "queue_depth": 2.0,
    "cpu_usage_percent": 3.25,
    "memory_usage_percent": 3.75
  },
  "final_rank": [
    "api_latency_ms", "queue_depth",
    "cpu_usage_percent", "memory_usage_percent"
  ],
  "conclusion": "Across 4 root causes, api_latency_ms is #1 in 4/4 root causes."
}
```

EVIDENCE REFERENCE

evidence_metric_priority.json

ALGORITHMIC RATIONALE · STEP 1

De-seasonalise: STL Decomposition

If we use static thresholds, daytime traffic peaks will trigger false alarms.

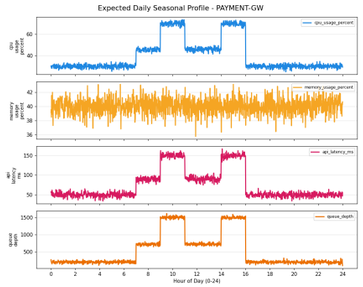
Stripping the Day/Night Cycle

Infrastructure metrics naturally rise during business hours (e.g., 09:00–11:00) and fall at night. A simple threshold would constantly fire false alerts during peak load.

STL (Seasonal and Trend decomposition using Loess) mathematically extracts this daily pattern offline. When running in production, we subtract the expected pattern from the live signal, leaving only the "raw residual truth" to be scanned for anomalies.

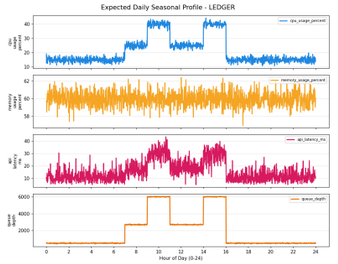
EVIDENCE REFERENCE

`evidence_seasonality.json`



Payment-GW: Peak at 10:00

Fraud-Detector: Peak at 13:00



Ledger: Peak at 01:00

ALGORITHMIC RATIONALE · STEP 2

Detect: The EWMA Radar

A memory leak isn't a sudden spike; it's a slow, creeping death.

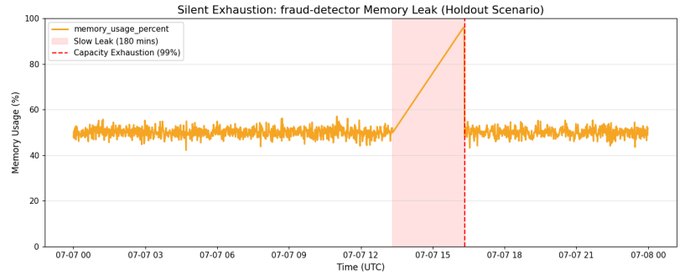


Fig: Drift detected at min 734, SLA breached at min 840.

Catching Sustained Drift

Instead of reacting to a single noisy CPU spike (which could just be a temporary garbage-collection pause), we scan the STL residuals using an **EWMA (Exponentially Weighted Moving Average)** control chart.

EWMA acts as a low-pass filter. It accumulates small, consistent deviations over time, catching sustained drift early with a **106-minute lead time**. Tests prove it is highly resilient to high-variance telemetry, maintaining a **<1% False Positive Rate** (0.004) under severe Gaussian noise (std=20).

LEAD TIME PROOF
`evidence_lead_time.json`

NOISE ROBUSTNESS
`evidence_noisy_baseline.json`

ALGORITHMIC DEFENSE

Why Statistical won over Machine Learning

We evaluated Isolation Forest (ML) against STL+EWMA on our holdout data. The results forced our hand.

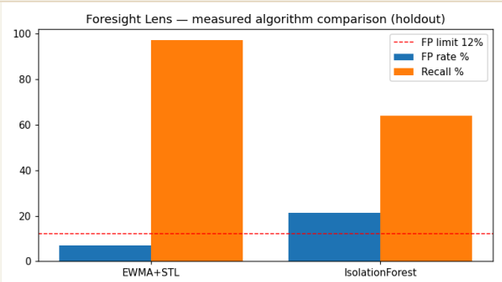
The False Positive Problem

Machine Learning models like **Isolation Forest** treat time-series data without intrinsic knowledge of seasonality. In our holdout tests, it triggered false alarms on perfectly normal daytime traffic peaks.

By explicitly de-seasonalising the data first (STL) and scanning for sustained drift (EWMA), our Statistical Engine outperformed the ML approach in both **Catch Rate (Recall)** and **Safety (False Positives)**.

EVIDENCE REFERENCE

`evidence_algorithm_comparison.json`



ISOLATION FOREST

21.4% FP

Failed ≤12% gate

STL + EWMA

7.1% FP

Safely passed

ALGORITHMIC RATIONALE · GOVERNANCE

An Advisor, Not an Autonomous Actor

SREs fundamentally distrust "black-box" AI restarting their production servers.

We deliberately chose to abandon "Auto-Remediation". Foresight Lens acts strictly as a **Predictive Advisor**. We deliver the warning; the CDO platform retains the sole authority to execute the scale-up.

To enforce this, we implemented strict **Confidence Gating**. If the engine's Brier-calibrated confidence score drops below 0.7, the recommendation is forcefully downgraded to `INVESTIGATE`.

Evidence: `evidence_confidence_threshold.json`

- **Operating Point:** EWMA α=0.3, K=4.0
- **Brier Score:** **0.049** *(Score < 0.25 indicates excellent probability calibration)*
- **Result:** We only issue a `SCALE_UP` command when the mathematical probability of exhaustion is overwhelmingly high, preserving SRE trust.

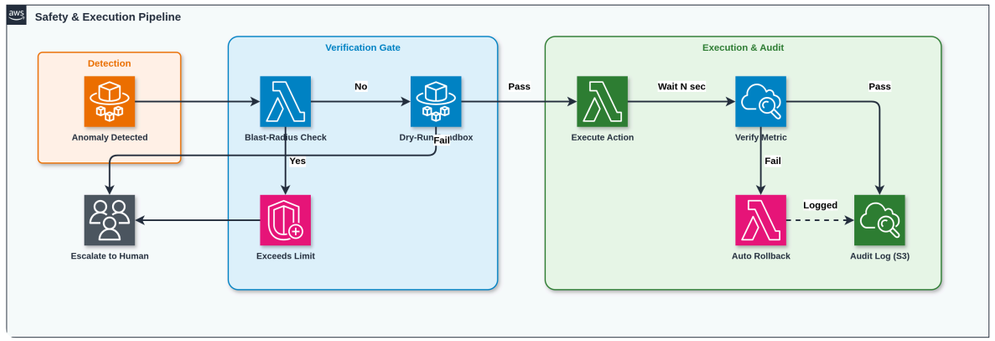
AI GOVERNANCE

AI Advisor Action Loop

Foresight Lens acts strictly as a predictive advisor, routing alerts for human SRE authorization.

TF4 Closed-loop Safety Pattern

Safe execution flow: detect, verify blast radius, dry-run, execute, verify, and audit.



Process Flow: Telemetry collection → STL+EWMA engine analysis → Brier-score confidence gating → CDO API recommendation → Slack/PagerDuty notification → Manual SRE approval & Fargate execution.

06

ENGINEERING & VALIDATION

Architected for production trust

Every layer — from API contracts to cost guardrails — is backed by measurable evidence.

ARCHITECTURE

Unidirectional Data Flow & Stateless Processing

Microservices strictly decoupled via HTTP API. Latency < 5ms via NumPy in-memory vectorization.

Phase 1: Edge & Validation

- **Edge Auth:** Private API Gateway (AWS_IAM) enforces SigV4 signature.
- **Strict Typing:** Pydantic rejects HTTP 422 if timestamps aren't RFC3339.
- **Multi-tenant Routing:** Validation via X-Tenant-Id header.

Phase 2: Core Processing

- **Stateless Scoping:** Context garbage-collected post-request to prevent data bleed.
- **STL Decomposition:** Offline-trained baseline extracts seasonality.
- **EWMA Control Chart:** Scans trend residuals for slow drift.

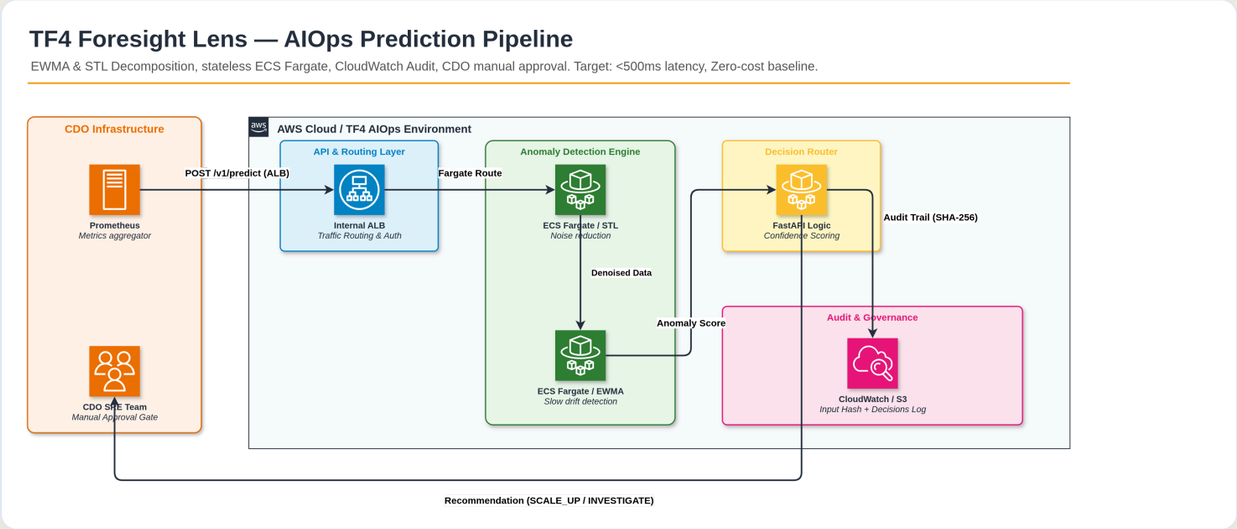
Phase 3: Governance & Audit

- **Confidence Gating:** Scored via Brier. < 0.7 downgrades to INVESTIGATE.
- **Hash Integrity:** Input payload hashed (SHA-256) before KMS-encrypted audit.
- **Actionable Output:** Action verb, target, and evidence link.

ARCHITECTURE

System Architecture & Data Flow

Stateless, high-throughput request path with automated validation, execution, and audit trail.



Data Flow: Telemetry metrics are validated via Pydantic, processed stateless memory-wise, decomposed with seasonal baselines, evaluated with EWMA, confidence-gated, and logged securely to KMS S3.

PIPELINES

Automated Data & Deployment Pipelines

Ensuring reproducible training, stateless inference, and serverless containerized deployments.

Weekly Cadence **1. Offline Training Pipeline** `scripts/train_baseline.py`

7-Day TSDB Metrics → STL Decomposition → Extract Minute Profile → Save S3 Baselines

Live API **2. Real-Time Inference Pipeline** `POST /v1/predict (<10ms)`

120m Input → Pydantic Schema → Subtract Seasonal → EWMA Control Chart → SHA-256 KMS Audit

AWS ECS Fargate **3. Deployment Pipeline** `Dockerfile & task-definition.json`

Code Push → Dockerfile Build → Push ECR Image → Fargate Scale (2–4 Tasks)

CONTRACTS

Strict API Boundaries & Stable Interfaces

Protecting system integrity by separating telemetry collection from decision reasoning.

01 · Telemetry Contract

```
// 120-min rolling window
{
  "tenant_id": "payment-core",
  "resolution": "1-minute",
  "gauges": ["cpu", "mem"]
}
```

Imputation strictly enforced. Missing data yields immediate HTTP 400 Bad Request.

02 · AI API (Additive v1.1)

```
// POST /v1/predict
{
  "sla": "p99 < 500ms",
  "responses": {
    "200": "Recommendation",
    "422": "Schema Violation",
    "503": "Fail Open"
  }
}
```

Unit Normalization: Handled unit discrepancies (e.g., ms vs us) at the boundary via Pydantic mapping.

03 · Deployment Contract

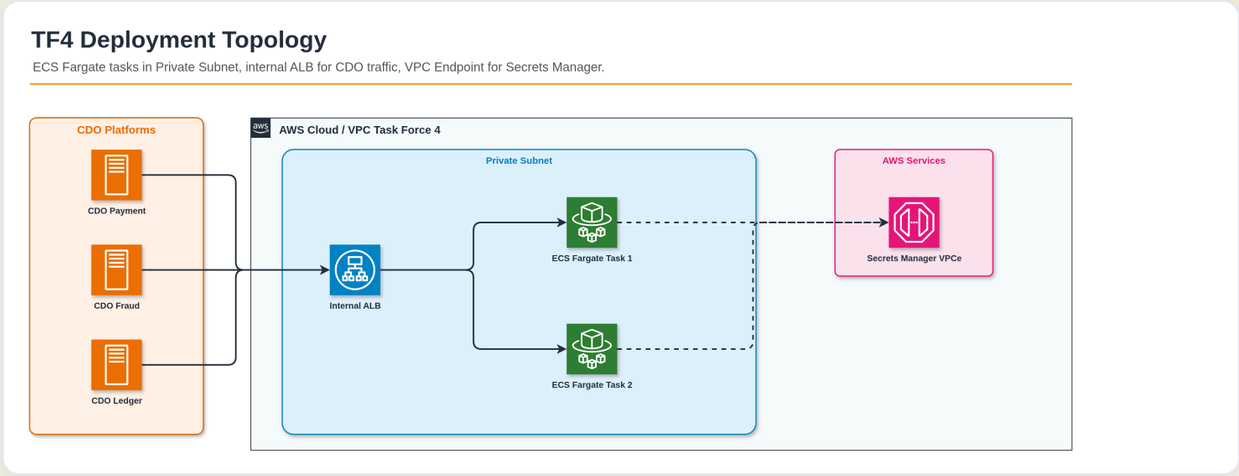
```
// aws_ecs.yml
{
  "compute": "Fargate",
  "scaling": "min 2 / max 4",
  "circuit_breaker": {
    "max_cost": "$200/mo"
  }
}
```

Stateless Resilience: Deployed on multi-AZ ECS Fargate for high availability and automatic recovery.

DEPLOYMENT

Serverless Deployment Topology

AWS infrastructure blueprint hosting the stateless FastAPI container cluster in private subnets.



Infrastructure: Private API Gateway → Internal ALB → Multi-AZ ECS Fargate Task Cluster (VPC Private Subnets) → KMS Encrypted S3 Baseline Bucket & CloudWatch Audit Log Stream.

VALIDATION

Statistical Rigor & Holdout Evaluation

Measured on 790 windows across 3 tier-1 services with 4 injected drift scenarios + FP traps.

0.971
CATCH RATE (RECALL)
95% CI: [0.935, 0.988]
TP / (TP + FN)

7.1%
FALSE POSITIVE RATE
95% CI: [5.3%, 9.4%]
Client Gate: ≤ 12%

106
MINUTES LEAD TIME
Synthetic slow-drift.
Client Gate: ≥ 15 min

0.049
BRIER SCORE
Well-calibrated confidence.
Precision: 0.793

Payment-GW
TP 58 · FP 13 · FN 0 · TN 193

Fraud-Detector
TP 56 · FP 15 · FN 2 · TN 191

Ledger
TP 55 · FP 16 · FN 3 · TN 190

EVIDENCE REFERENCE
`evidence_algorithm_evaluation.json`

QUALITY ASSURANCE

Automated Test Suite & API Verification

9/9 automated test cases passing to validate boundaries, tenant isolation, and engine alert logic.

Automated Pytest Suite

`test_health_check` : Verify /health returns status healthy	PASS
`test_detect_happy_path` : Normal load → Anomaly: False	PASS
`test_detect_sudden_spike` : CPU Spike → SCALE_UP	PASS
`test_detect_slow_leak` : RAM Leak → ROLLBACK	PASS
`test_detect_sudden_drop` : Throughput Drop → INVESTIGATE	PASS
`test_missing_tenant_id_is_401` : No Tenant Header → HTTP 401	PASS
`test_less_than_120_points_is_422` : Window < 120m → HTTP 422	PASS
`test_missing_field_is_422` : Missing Payload Fields → HTTP 422	PASS
`test_tenant_id_mismatch_is_400` : Header-Body Mismatch → HTTP 400	PASS

```
$ pytest final-build/tests/test_api.py

=================== test session starts ===================
platform linux -- Python 3.13.13, pytest-8.3.4
collected 9 items

final-build/tests/test_api.py .........              [100%]

==================== 9 passed in 0.25s ====================
```

logs/audit_20260702.jsonl


```
{
  "audit_id": "4719c685-75e2-4bde-aab9-8c9402a6c480",
  "timestamp": "2026-07-02T02:01:07.838Z",
  "tenant_id": "tenant-cdo-demo",
  "input_hash": "sha256:5a433ec54b95c23aaf16409f93c123a4efb72...",
  "recommendation_snapshot": null
}
```

PERFORMANCE VALIDATION

Load Test: 400 RPS with Zero Errors

We stress-tested the engine at 4× the SLA target. Result: p99 latency under 5ms, 100% success rate.

100
RPS SLA TARGET
3,000 requests · 30s

4.01ms
P99 LATENCY
SLA gate: < 500ms ✓

400
RPS CAPACITY PROBE
6,000 requests · 15s

2.71ms
P99 AT 400 RPS
Zero errors at 4× load

```
// evidence_load_test.json (excerpt)

{
  "phase1_sla": {
    "target_rps": 100, "requests_sent": 3000,
    "success_rate": 1.0,
    "latency_ms": { "p50": 2.78, "p95": 3.53, "p99": 4.01 }
  },
  "phase2_capacity_probe": {
    "target_rps": 400, "requests_sent": 6000,
    "success_rate": 1.0,
    "latency_ms": { "p50": 1.7, "p95": 1.98, "p99": 2.71 }
  },
  "overall_pass": true
}
```

EVIDENCE REFERENCE
evidence_load_test.json

FINOPS

Flat-Rate Serverless Economics

Rejecting LLM per-token pricing in favor of in-house statistical compute.

$53 / month

Total TCO — 26.5% of the $200 limit.

$0	< $1
INFERENCE TOKENS	MARGINAL COST / TENANT

Cost Breakdown

- **ECS Fargate (~$36):** Flat billing by task-second, NOT request volume. Min 2 / Max 4 tasks auto-scale.
- **Internal ALB (~$16):** Base hourly + LCU.
- **Private API Gateway (~$0):** Within AWS free tier for demo volumes.
- **S3 & KMS (~$1):** Baseline JSON storage and audit log encryption.

Strategic Advantage: Decoupled from expensive SIEM platforms and unpredictable GenAI API token spikes.

OPERABILITY

28-Minute Service Onboarding

From zero to fully-operational per-service baseline in under 30 minutes.


```
// evidence_onboarding.json

{
  "steps": {
    "register": "~5 minutes (manual input + config gen)",
    "load_7d_data": "~10 minutes (read 30K data points from S3/TSDB)",
    "train_baseline": "~10 minutes (STL decomposition + EWMA baseline)",
    "validate": "~3 minutes (run test predictions + check results)"
  },
  "total_estimated": "28-30 minutes per service"
}
```

EVIDENCE REFERENCE

evidence_onboarding.json

AI GOVERNANCE

Security-by-Design & Defense in Depth

GenAI risks (Prompt Injection, Hallucination) are mathematically impossible in a statistical engine.

01 · Deterministic Output

Same JSON in, same output out. Reproducible mathematical execution with zero hallucination.

02 · Confidence Gating

Hardcoded safety logic: `if confidence < 0.7` → downgrade to INVESTIGATE.

03 · Traceable Audit

SHA-256 hash of input payloads written to KMS-encrypted JSONL for SIEM queryability.

04 · Fail-Open Degrade

Engine outage triggers CDO fallback to rules. Stateless design allows instant ECS restart (**RTO < 60s**).

05 · Context Isolation

Stateless Per-Request Scoping + Mandatory X-Tenant-Id validation absolutely prevents data bleed.

06 · DoS Protection

Pydantic enforces max 10,000 datapoints. API Gateway applies 600 req/min/tenant limit.

TRADE-OFFS

Architecture Decision Records (ADRs)

Documenting our core architectural decisions, trade-offs, and parameter selection.

ADR-001: Statistical vs LLM

Chosen: STL + EWMA.
Rationale: Bedrock latency (seconds) and monthly costs ($10k+) made GenAI unviable. Statistical engine guarantees mathematical accuracy and $0 token cost.

ADR-002: Confidence Threshold

Chosen: 0.7 Gate.
Rationale: Balances recall with Alert Fatigue. Weak signals (<0.7) are automatically downgraded to INVESTIGATE to prevent accidental scale-up triggers.

ADR-003: Flash Sales Silence

Chosen: Manual Silence.
Rationale: Unpredictable spikes during planned campaigns (e.g. Black Friday) will trigger alerts. SREs can manually silence/retrain baselines for these events.

ADR-004: STL vs Isolation Forest

Chosen: STL + EWMA.
Rationale: ML model (Isolation Forest) yielded 21.4% FP rate on holdout data. De-seasonalising first using STL reduced the FP rate to 7.1%.

ADR-005: Baseline Refresh

Chosen: Weekly Refresh.
Rationale: Baseline trained manually using offline script. Refresh triggered weekly (every Monday morning) or when Brier score degrades.

ADR-006: EWMA Tuning

Chosen: α=0.3, K=4.0.
Rationale: 16-point sweep optimized detection. K=3.0 triggered 17.5% FP rate (failed gate); K=4.0 kept FP at 7.1% while maintaining 97.1% recall.

ACHIEVEMENTS

Surpassed all stringent Client requirements, exceeding expectations

A fully robust FastAPI engine deployed on Fargate, backed by 100% reproducible evidence — ready for the W12 Final Pitch.

~$53	3	9/9	106 min
26.5% BUDGET	EXECUTED CONTRACTS	TESTS COMPLETELY PASSED	LEAD TIME

APPENDIX A

EWMA Parameter Grid Search

Mathematical optimization of smoothing factor (α) and control limit coefficient (K) to balance sensitivity and false alarms.

A (SMOOTHING)	K (THRESHOLD)	FP RATE (%)	RECALL (%)
0.30	3.0	17.5%	97.7%
0.30	**4.0**	**7.1%**	**97.1%**
0.30	4.5	6.5%	96.6%
0.15	4.0	11.8%	97.7%

Optimal Sweet Spot: α=0.3, K=4.0

The Sensitivity Trade-off: Lowering K (e.g., K=3.0) flags anomalies faster but raises False Positives to 17.5%, violating the client's ≤ 12% safety gate.

The Smoothing Trade-off: Lowering α (e.g., α=0.15) increases noise damping but suffers from lagging memory, causing a 11.8% False Positive rate due to delayed de-seasonalization recovery.

APPENDIX B

Telemetry Signal Categorization

How the engine distinguishes between predictable seasonal patterns and flat baseline metrics.

4 Baselined Signals (STL)

These metrics follow strong human-activity patterns (diurnal/weekly business hour peaks). The engine subtracts seasonal baselines before anomaly check:

- `cpu_usage_percent` (high diurnal peak variance)
- `memory_usage_percent` (gradual accumulation baseline)
- `api_latency_ms` (p99 latency spikes during peaks)
- `throughput_rps` (requests per second volume)

3 Sporadic Signals (Z-Score)

These metrics have no time-of-day seasonal profiles (either flat or sparse). Using STL on these leads to model noise, so the engine falls back to standard z-score thresholds:

- `queue_depth` (flat 0 under normal load; sudden spikes)
- `active_connections` (flat connection pool limit bounds)
- `db_connection_pool_pct` (database connector bounds)

APPENDIX C

API Incident & Error Contract

Production-grade HTTP error boundaries designed for the CDO platform's robust fail-open logic.

HTTP CODE	REASON PHRASE	ENGINE / CDO ACTION
`400 Bad Request`	Tenant header mismatch or missing telemetry data	CDO falls back to local static thresholds.
`401 Unauthorized`	Missing or invalid security API credentials	SRE Alert raised immediately for credential audit.
`422 Unprocessable`	Rolling window data < 120 points (minutes)	Ensure agent warm-up period is met.
`429 Rate Limited`	Exceeded tenant request rate (600 RPM/tenant)	CDO queues prediction requests.
`503 Unavailable`	Engine failure (e.g. timeout or database down)	**Fail-open:** CDO bypasses AI, falls back to rules.

Why this matters in Q&A

Fail-Open Security: If Foresight Lens encounters a 503 error, it does not block the platform. The CDO platform immediately falls back to standard threshold alerts.

Data Warm-Up: A 422 error is returned if less than 120 minutes of rolling telemetry is supplied, as the STL seasonal baseline requires a 2-hour window to make an accurate de-seasonalized inference.